Trabalho para P1 – 2026-1

versão 1.3.0 (09/04/2026) – Ajustes gerais, na cor verde.
versão 1.2.0 (08/04/2026) – Ajuste sobre o uso de frameworks CSS.
versão 1.1.0 (03/04/2026) – Ajustes no escopo.
versão 1.0.0 (02/04/2026) – Inicial.

Tarefa: Construir e apresentar uma aplicação web utilizando controles de qualidade, de acordo com o conteúdo abordado na disciplina e os detalhes descritos neste documento.

Datas:

- Entrega : 30/04 18:00
- Apresentações : 30/04 18:10 e 07/05 18:10 (ordem por sorteio)

Apresentação:

1. Duração: 25 minutos;
2. **Obrigatória.** Trabalhos entregues sem apresentação não pontuarão;
3. Com arguição do professor durante a apresentação (tempo descontado da duração);
4. *Durante a apresentação, o professor poderá solicitar a qualquer apresentador que **construa, na hora**, um ou mais artefatos relacionados ao sistema, quaisquer que sejam (front-end, back-end, testes etc.), **sem uso de consultas ou ajuda** (inclusive do próprio código da aplicação). A não construção do solicitado de forma satisfatória implicará em penalização na nota;*
5. Deve expor o projeto arquitetural criado para a aplicação;
6. Deve expor a aplicação construída em pleno funcionamento;
7. Deve expor todos os controles de qualidade adotados;
8. Com slides, *offline*, que devem ter pelo menos a seguinte estrutura:
    a. **Capa**, com nome da disciplina, período, nome do(s) aluno(s) e e-mail(s) institucional(is);
    b. ~~**Problema e**~~ **Modelo Essencial**, com o Diagrama de Classes (UML) do Modelo Essencial, que cubra todo o Minimundo;
    c. **Arquitetura**, com um Diagrama de Classes (UML) Parcial que exemplifique parte da arquitetura construída para a aplicação (ex. Repositórios, Serviços, Controladoras, Visões, etc. que foram criados para a funcionalidade principal apenas);
    d. **Exposição da Aplicação**, em que a aplicação é mostrada em execução, sem uso de slides;
    e. **Código e Controles de Qualidade**, exposição, sem uso de slides, do código-fonte do projeto e dos controles de qualidade adotados, tal como testes automatizados e ferramentas de análise estática, em execução;
    f. **Referências**, com referências utilizadas para a construção da aplicação ou da apresentação. Devem usar o formato da ABNT;
    g. **Perguntas**, com o(s) e-mail(s) institucional(is) do(s) aluno(s);
9. Os slides **não devem** conter código-fonte de qualquer natureza ou qualquer explicação sobre a estrutura do código;
10. Os slides devem ser entregues em PDF junto à entrega do código da aplicação, até a data e horário da entrega do trabalho.

Avaliação:

- É individual;
- Considerará o domínio do assunto e o montante de trabalho realizado por cada indivíduo no projeto;
- Considerará todos os aspectos do projeto, *tais como* arquitetura, controle de qualidade, boas práticas de construção, *front-end*, *back-end*, qualidade da interface de usuário e funcionalidades entregues;
- Arguições durantes os encontros, nas aulas, poderão ser consideradas.

Entrega:

- No branch "main" do repositório do GitLab **criado pelo professor** para o grupo.

Repositório:

1. Será avaliado **ao longo do projeto** (frequência de uso, organização, boas práticas etc.). Por exemplo, deve-se enviar código *frequentemente* e evitar realizar *commits* com conteúdo pouco relevante;
2. **Não deve** incluir a pasta /**node_modules**, nem pastas ou arquivos que possam ser gerados a partir do processo de instalação, compilação ou build (ex. /**dist**, /**output**, /**vendor**);
3. Deve incluir o arquivo de apresentação, "/**apresentacao.pdf**";
4. Deve incluir todos os arquivos de **código-fonte** necessários;
5. Deve incluir um arquivo /**readme.md** com:
    a. **nome completo** do(s) aluno(s);
    b. **comandos** necessários para pôr o projeto em funcionamento;
    c. **referências** bibliográficas ou locais de onde foram obtidas imagens, estilos CSS, ou outros recursos aplicáveis. Indicar o que foi obtido de cada local.

Construção da aplicação:

1. Deve ser realizada tanto **durante as aulas** quanto fora delas. Durante as aulas, deve-se procurar sanar todas as dúvidas com o professor, tais como **requisitos**, arquitetura, interface de usuário e implementação. É imprescindível participar das aulas e validar (pegar *feedback*) com o professor sobre o que está sendo construído.

2. Não deve utilizar IA ou geradores de aplicação/módulos/código **de nenhuma natureza**. A aplicação e **todos** os seus recursos devem ser construídos **manualmente**. **Imagens e trechos de conteúdos CSS** podem ser *obtidos da Internet*, tomando-se a precaução de documentar o respectivo site no README.md do projeto. **Atenção**: Detectores de código gerado por IA serão utilizados. No caso de **quaisquer suspeitas** do uso de IA, haverá descontos na nota, podendo até haver a desconsideração total do trabalho – mesmo que não tenha sido utilizado em sua totalidade.

3. Não deve utilizar quaisquer trechos de outros trabalhos. O uso será caracterizado com plágio acadêmico, havendo de penalizações na nota à desconsideração total do trabalho, mesmo que haja plágio parcial.

4. Haverá arguição oral sobre qualquer conteúdo do trabalho. Conteúdos não explicados satisfatoriamente serão **descontados** da nota do trabalho, podendo até perder toda a sua pontuação;

5. Os **diagramas** (nos slides ou fora deles) e o **código-fonte** não devem incluir comentários que expliquem o que fazem ou suas responsabilidades;

6. Deve empregar a arquitetura de separação de responsabilidades MVP, tanto no *front-end* quanto no *back-end*;

7. Tecnologias do *Front-end*:
    a. Utilizar o gerenciador de pacotes PNPM;
    b. Utilizar a ferramenta de *build* Vite;
    c. Utilizar o *framework* de testes unitários/integração ViTest;
    d. Utilizar o *framework* de testes *end-to-end* Playwright;
    e. Implementar a aplicação em linguagem TypeScript;
    f. **Não** utilizar bibliotecas ou *frameworks* JavaScript/TypeScript para construção de interface de usuário (ex. por componentes), tal como ReactJS, VueJS, etc;

g. Utilizar um *framework CSS* para acelerar a estilização da interface de usuário de forma responsiva, tal como Bulma, Bootstrap 5 (sem jQuery) ou Material Design Lite; Admite-se o uso de um framework CSS baseado em Tailwind, como o DaisyUI, *apenas* se: (*a*) todos do grupo detiverem experiência prévia uso de Tailwind; **e** (*b*) todos concordarem com seu uso. Procure substituir, sempre que possível, o uso de várias classes CSS do framework, por uma classe CSS da sua aplicação (utilize @apply para isso).
h. Ao criar uma *Single-Page Application* (SPA), pode-se utilizar uma biblioteca para tratar rotas, como page.js ou crayon.
i. Pode-se utilizar alguma biblioteca de código para gerar gráficos (*charts*) para a aplicação (vide algumas sugestões), caso necessário.

8. Tecnologias do *Back-end*:
   a. Utilizar PHP 8 e o gerenciador de pacotes Composer;
   b. Utilizar a biblioteca de testes unitários/integração Kahlan (preferencialmente) ou Pest;
   c. Utilizar a ferramenta de Análise Estática PHPStan com nível 6, no mínimo;
   d. Utilizar o banco de dados MariaDB;
   e. Pode-se utilizar uma biblioteca/framework para tratar rotas RESTful no back-end, tal como a phputil/router ou Slim, com *middlewares* para CORS.

Quaisquer dúvidas devem ser dirimidas com o professor o mais breve possível.

Enunciado:

I. Minimundo

Em um universo alternativo, o CEFET Nova Friburgo deseja construir uma loja virtual para vender itens – tais como bonés, canecas, camisetas, blusas de frio, calças, pulseiras e broches – de seus cursos. Nessa loja, a ser chamada de Cefet Shop, deve haver uma única moeda e meio de pagamento: o Cefetin. Um Cefetin (C$) equivale a um Real (R$). Estudantes ganham Cefetins quando concluem disciplinas, em cada período do curso. Os Cefetins ganhos equivalem a 5 (cinco) vezes a sua média final (MF) em cada disciplina. Por exemplo, se um aluno consegue 8.0 de MF em uma disciplina, ganha 40 Cefetins. Se cursa 5 disciplinas e consegue 8.0 de MF em todas elas, ganha 200 Cefetins. Com os Cefetins que ganha, pode comprar itens no site. Ao cursar uma disciplina pela segunda vez ou mais, ganha apenas 20% do valor que ganharia.

A tela inicial do Cefet Shop deve exibir os itens mais vendidos, em forma de *cards*, de forma paginada – com 6 itens por página. Portanto, a tela inicial deve exibir os 6 itens mais vendidos, a segunda página os próximos 6 itens, e assim por diante. Cada item do *card* deve ser exibido com foto, descrição e preço de venda, além do botão "Detalhes". Caso um item esteja em promoção, o percentual de desconto (que pode variar de 5 a 20%) deve ser exibido. Nesse caso, o preço sem o desconto deve aparecer tachado (riscado), em tamanho menor, acima do preço de venda.

Em um item da tela inicial, ao clicar em "Detalhes", em sua foto, ou em sua descrição, uma tela que fornece detalhes sobre o item deve ser exibida. Essa tela de detalhes do item deve conter, além dos dados do item descritos anteriormente, o seu período de lançamento (ex. "2026-1"), um texto que descreve os detalhes do item, uma caixa de seleção para selecionar a quantidade desejada, e os botões "Adicionar ao Carrinho" e "Ir para o Carrinho". A opção para selecionar a quantidade do item e o botão "Adicionar ao Carrinho" devem ficar desabilitados e ocultos, caso o estoque do item seja zero (0), sendo exibido o texto "Esgotado" ao usuário. Se

houver menos de 10 unidades no estoque, a caixa de seleção deve exibir de 1 até o valor disponível em estoque. Já se houver 10 unidades ou mais, deve exibir os valores de 1 a 10. O botão "Ir para o Carrinho" deve redirecionar o usuário para visualizar o Carrinho de compras. A opção "Adicionar ao Carrinho" deve adicionar o item ao carrinho de compras e finalizar a exibição da tela de detalhes do item.

O Cefet Shop deve mostrar, em seu topo e de forma fixa, um ícone que representa o Carrinho. Cada item diferente acrescentado ao carrinho deve fazer com que um *badge* contendo a contagem de itens seja exibido junto ao ícone. Esse *badge* não deve for exibido se o Carrinho estiver vazio.

Na tela do Carrinho, devem ser exibidos os itens adicionados, do mais recente para o mais antigo, além de um Total (C$) e um botão "Finalizar Compra". Ao lado de cada item deve haver um Subtotal (multiplicação do preço pela quantidade adicionada). Essa quantidade deve poder ser ajustada, assim como na tela de detalhes do item. Ao ajustar a quantidade, o subtotal e o total devem ser atualizados. Em cada item deve haver um botão "Remover" para retirar o item completamente do Carrinho, após confirmação do usuário. Ao não haver itens no Carrinho, deve ser exibida a mensagem "Seu carrinho de compras está vazio." e a opção "Finalizar Compra" deve ser desabilitada e ocultada.

No Carrinho, acionar a opção "Finalizar Compra" deve fazer verificar se o usuário (aluno) está logado. Se não estiver logado, o sistema deve redirecionar o usuário para a tela de *login*. Ao concluir o *login*, o usuário deve ser redirecionado ao Carrinho de Compras, para que possa finalizar a compra, do ponto em que estava. Assim, estando logado, o sistema deve verificar o saldo de Cefetins do usuário e debitá-lo no valor da compra. Caso o usuário possua menos Cefetins que o valor da compra, o sistema deve informar que o usuário não possui saldo suficiente para concluir a compra e sugerir que remova um ou mais itens da compra. Sendo possível concluir a compra, o estoque dos itens deve ser verificado, de acordo com a quantidade atualmente disponível em estoque. Caso algum item não contenha estoque suficiente, o sistema não deve finalizar a compra, deve alterar a quantidade do respectivo item para o disponível em estoque ou removê-lo do Carrinho e avisar o usuário da mudança (por falta de estoque). Assim, para finalizar a compra, o usuário precisará acionar a opção “Finalizar Compra” novamente, estando ciente dos ajustes. Havendo estoque dos itens e saldo na conta do usuário, o sistema deve salvar a compra, atualizar o estoque dos itens envolvidos, atualizar o saldo do usuário e informar ao usuário sobre o número da sua compra. O sistema também deve informar ao usuário que este deverá passar no Almoxarifado do CEFET das 9h às 18h para resgatar os itens comprados, de posse de seu número de compra.

A tela de Login deve permitir que um usuário acesse o sistema utilizando sua matrícula ou seu e-mail institucional, mais uma senha de acesso.

A secretaria mantém o cadastro de todos os alunos automaticamente, incluindo a atualização dos saldos a cada final de período letivo.

II. Escopo para a P1:

1) **Tela inicial.**
2) **Tela de detalhes do item.**
3) **Carrinho de Compras**, sem finalização da compra/login.

III. Observações:

- Não é necessário haver cadastros de nenhuma natureza na aplicação. Deve-se preencher o banco de dados com todos os dados necessários.
- <u>Todos do grupo devem fazer um pouco de cada funcionalidade</u>, envolvendo *front-end*, *back-end* e controles de qualidade.